

Interview Prep Agent — Architecture Document

1. Purpose

This document describes the architecture of the Interview Prep Agent application. It provides a high-level overview of system components, how they interact, the models used, and the rationale behind the key design decisions.

2. Scope

The system under discussion supports two primary modes:

- **Question Generation Mode:** The system uses a large-language model (LLM) to generate interview questions based on a user-provided role, topic or resume.
- **Interactive Interview Mode:** The system presents generated questions to the user, captures their answer, uses a fine-tuned local LoRA model to score the answer, and uses the LLM again to provide feedback/model answer.
The document covers components, flows, technology stack, deployment, and decision rationale. It does *not* cover detailed UI wireframes, dataset cleaning scripts, or each library's internal implementation.

3. Architectural Drivers

3.1 Functional Requirements

- FR1: Allow users to choose a topic (e.g., “Data Analyst: SQL & Pandas”) and number of questions.
- FR2: Generate questions dynamically from LLM.
- FR3: Present each question, collect answer, trigger scoring.
- FR4: Provide feedback and model answer for each question.
- FR5: Support uploading a resume or custom prompt (in generation mode).

3.2 Non-Functional Requirements

- NFR1: **Performance** – Real-time or near-real-time response for question generation and feedback.
- NFR2: **Scalability** – Able to handle many users (single instance may suffice initially).
- NFR3: **Maintainability/Extensibility** – Easy to plug in different LLMs or different scoring models.
- NFR4: **Security** – Secure API keys and credentials, user content confidentiality.
- NFR5: **Usability** – Clear UI, minimal friction in workflow.

3.3 Constraints & Principles

- Use of existing models: the architecture assumes one external LLM API (e.g., Gemini) and one local LoRA model.

- Prefer cloud-friendly, containerizable deployment (e.g., via Docker/Netlify for UI, backend as microservice).
- Modular separation of UI, orchestration logic, model-invocation logic, scoring logic.

4. High-Level System Context

- External LLM API (Gemini) – question generation and feedback.
- Local LoRA model – scoring engine (runs locally or containerised).
- Data store/session state – lightweight (e.g., in-memory or Redis) for storing generated questions/answers.
- Optional file store – for uploaded resumes or custom prompts.

5. Container/Module View

5.1 UI Module

- Built with Streamlit.
- Responsible for presenting user input (topic, number of questions), display of questions, capturing answers, invoking score/feedback flows.
- Maintains session state via `st.session_state`.

5.2 Orchestration Module

- Responsible for controlling flow: when user presses “Start Interview”, it sends prompt to LLM API, parses result, stores questions.
- For each question: triggers scoring and feedback workflows.

5.3 LLM Integration Module

- Encapsulates calls to the external LLM API: question generation, feedback generation.
- Contains templating for prompts.

5.4 Scoring Module

- Encapsulates the local LoRA model invocation (`LocalLoRAModel.score_answer`) for scoring answers.
- May wrap or manage model loading, caching, scaling.

5.5 Session/State Store

- E.g., in-memory or lightweight store to persist questions, answers, scores during a session.
- Ensures that UI can re-render when user interacts with different question scoring buttons.

6. Runtime/Interaction Flow

Actor: User

1. User selects mode → generation or interactive.
2. If interactive: user enters topic & number of questions → clicks Start.

3. Orchestration module sends prompt to LLM → receives numbered question list.
4. UI displays first question (and sequentially others).
5. User enters answer for Q1 → clicks Score.
6. Scoring module computes score → UI displays “LoRA Score: X/10”.
7. Orchestration module sends feedback prompt to LLM → receives model answer & feedback → UI displays it.
8. Steps 5-7 repeat for Q2... Qn.
9. Session ends

Sequence Diagram

user → UI → Orchestration → LLM/Scoring → UI

7. Deployment View

- **Web UI container:** Streamlit app; can be hosted on Netlify (for static part) or via a server (e.g., AWS ECS, Azure App Service).
- **Backend container(s):** orchestration + LLM calls + scoring service. Could be combined with UI depending on scale.
- **Local LoRA model:** may run on GPU/CPU machine; accessible via internal API endpoint.
- **Session store:** Redis or in-memory.
- **Optional persistent store:** PostgreSQL/MySQL for analytics.
- **External LLM API endpoint:** Cloud service (Gemini).

8. Technology Stack

- UI: Python, Streamlit
- LLM API: Gemini (via GeminiClient)
- Local scoring model: TinyLlama LoRA adapter (LocalLoRAModel)
- Session store: Python st.session_state, optional Redis
- Containerisation: Docker + optionally Kubernetes if scaling
- Hosting: Netlify for UI, or cloud provider for full stack
- Monitoring/logging: e.g., Prometheus + Grafana, or Cloud provider logs

9. Key Architectural Decisions & Rationale

Decision	Why	Alternatives Considered
Use external LLM (Gemini) for both question generation & feedback	Leverage state-of-the-art language model without building custom model from scratch	Build own open-source LLM (too heavy)
Use local LoRA scoring model (TinyLlama adapter)	Enables offline scoring, no dependency on external API for score, faster latency	Use LLM for scoring (costly, latency)
Session state via Streamlit st.session_state	Simplest for per-user session persistence in UI	External DB/store (overkill)
Modular separation (UI, orchestration, scoring, LLM)	Improves maintainability & extensibility	Monolithic block (would reduce clarity)

10. Quality Attributes & How They Are Addressed

- **Performance:** Local LoRA model scoring reduces latency compared to external calls.
- **Scalability:** Containerised modules allow horizontal scaling of UI/back-end.
- **Extensibility:** Plug-in architecture means future scoring or feedback models can be swapped.
- **Security:** API keys stored securely (via environment variables). User data (answers) may not be persisted unless explicitly.

11. Security & Privacy Considerations

- Store LLM and LoRA model credentials securely in environment variables.
- Ensure user answers are not persisted or are anonymised if stored.
- Secure endpoints (if accessible externally) via HTTPS, authentication if needed.
- Protect against injection attacks in prompts (sanitize user input for topics/prompts).

12. Limitations & Future Enhancements

- Currently, scoring is limited to the one LoRA model — might not cover all domains equally.
- Feedback latency depends on external API; caching or queueing might help.
- Future enhancements: multilingual support, adaptive difficulty, analytics dashboard for user improvement tracking.

13. Glossary & Acronyms

Acronym Definition

LLM	Large Language Model (e.g., Gemini)
LoRA	Low-Rank Adaptation — technique to fine-tune large models with fewer resources
UI	User Interface

Acronym Definition

API Application Programming Interface

Architectural Decision Records (ADR) log

- ADR-001: Choice of Gemini LLM for question generation
- ADR-002: Choice of TinyLlama LoRA for scoring